

EMBEDDED SYSTEM AND DESIGN

EXPERIMENT NO. 4

APPLIANCE CONTROLLER

Name: *Raphael S. Nano*

Course/Section: *CPE161P / AI*

Date of Performance: *December 5-6, 2023*

Date of Submission: *December 8, 2023*

Ma'am Cyrel Manlises
Instructor

DISCUSSION

Appliance controllers highlight a wide variety of features, ranging from timers to complex IoT devices capable of managing and optimizing the efficiency of household appliances. As they became an integral component in modern households, appliance controllers augment convenience, along with their ability to promote cost-effectiveness and energy efficiency. Hence, it is important to highlight the capabilities of an appliance controller and their processes of construction in order to further develop its functionality. With that said, an appliance controller system was constructed and replicated using Arduino Uno as the controller and the stepper motor, servo motor, passive buzzer, and LEDs as the devices being controlled.

The objectives of this experiment are as follows:

- 1. To demonstrate how an appliance controller works in real-time.*
- 2. To be able apply what you have learned in logic circuits and switching theory, microprocessors, and CPE160P.*
- 3. To be able to apply real-time embedded system using Arduino kit.*

The materials utilized for this experiment include the Arduino Uno, the infrared remote, TSOP1838 IR receiver sensor, 28BYJ-48 stepper motor, the SG90 servo motor, a passive buzzer, and three (3) LEDs.

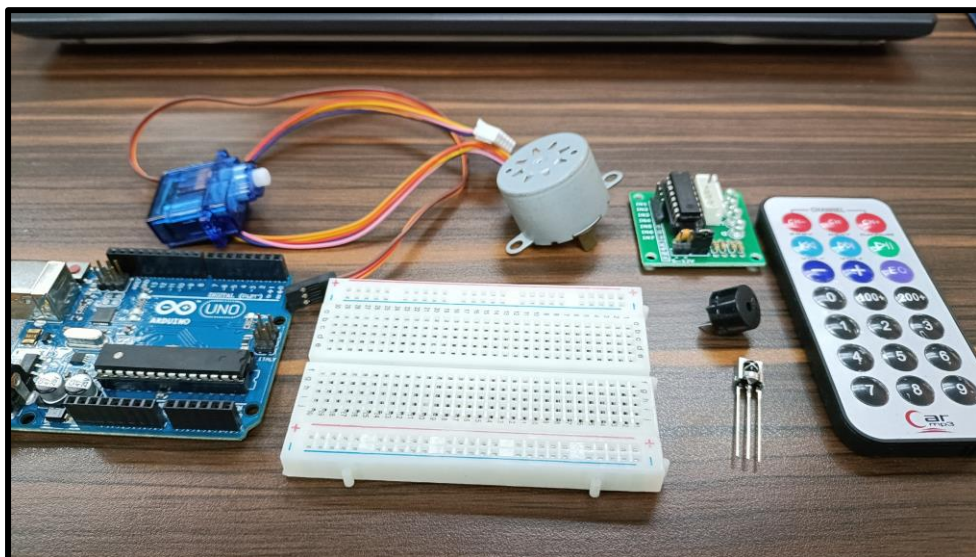


Figure 1. Major Components for the Experiment

For the external libraries, the IRremote.h, Stepper.h, and Servo.h was utilized for the remote, the stepper motor, and the servo motor respectively.

```
1  #include <IRremote.h>
2  #include <Stepper.h>
3  #include <Servo.h>
```

Figure 2. External Libraries Included

The primary objective for experiment 4 is to replicate an application controller that operates in real-time. The IR remote will be used as the controller, wherein it includes the control to every appliance (e.g., stepper motor, servo motor, buzzer, LEDs). The operations simply turns the appliance ON and OFF with the use of the numbers on the remote and the pause/play button.

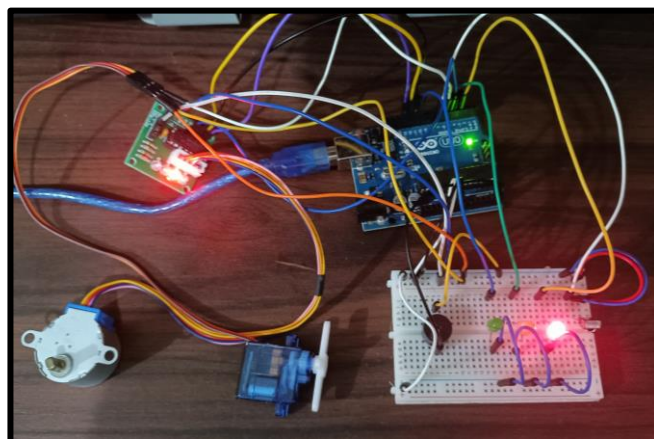


Figure 3. Final Output of the Experiment

An optimized switch case inside a remote() function was utilized to determine the value of the button pressed in the remote. Using the function IrReceiver.decodeIRData.command, the values were obtained experimentally. For button 0, an integer of 22 is decoded. 12 for button 1, 24 for button 2, 94 for button 3, 8 for button 4, 28 for button 5, and 67 for the pause/play button.

```
28 void remote() {
29     if (IrReceiver.decode()) {
30         switch (IrReceiver.decodedIRData.command) {
31             case 22:
32                 irval = 0;
33                 break;
34             case 12:
35                 irval = 1;
36                 break;
37             case 24:
38                 irval = 2;
39                 break;
40             case 94:
41                 irval = 3;
42                 break;
43             case 8:
44                 irval = 4;
45                 break;
46             case 28:
47                 irval = 5;
48                 break;
49             case 67:
50                 irval = 10;
51                 break;
52         }
53         IrReceiver.resume();
54         Serial.println(irval);
55     }
56 }
```

Figure 4. Switch Case for IR receiver

Since button 1 is assigned to the stepper motor, the stepper motor continuously rotates as long as the pause/play button is not yet pressed.

```
126     if (irval == 1) {  
127         while (irval != 10) {  
128             stepper.step(steps);  
129             remote();  
130         }
```

Figure 5. Code for stepper motor

Button 2 is assigned to the servo motor, wherein it writes 180 degrees and 0 degrees every after 500 milliseconds.

```
131     } else if (irval == 2) {  
132         while (irval != 10) {  
133             servo.write(180);  
134             msdelay(500);  
135             servo.write(0);  
136             msdelay(500);  
137             remote();  
138         }
```

Figure 6. Code for servo motor

The passive buzzer is assigned to button 3, and in its code, a separate playTone() function is created.

```
139     } else if (irval == 3) {  
140         while (irval != 10) {  
141             playTone(1000, 100);  
142             remote();  
143         }
```

Figure 7. Code for activating passive buzzer

For the passive buzzer, its frequency was controlled and its duration was specified in order to trigger properly. The pin where the buzzer is connected is simply triggered HIGH and LOW in order to turn it ON and OFF, respectively.

```
58 void playTone(int frequency, int duration) {
59     for (long i = 0; i < duration * 1000L; i += 2 * frequency) {
60         digitalWrite(A4, HIGH);
61         delayMicroseconds(frequency);
62         digitalWrite(A4, LOW);
63         delayMicroseconds(frequency);
64     }
65 }
```

Figure 8. playTone() function

For the last component, the LEDs, a pattern of lights was constructed wherein the LEDs turn ON through a process of 1-2-3, ALL-ALL, 3-2-1. It is assigned to button 4.

```
76 void LED() {
77     digitalWrite(3, HIGH);
78     msdelay(50);
79     digitalWrite(3, LOW);
80     msdelay(50);
81     digitalWrite(4, HIGH);
82     msdelay(50);
83     digitalWrite(4, LOW);
84     msdelay(50);
85     digitalWrite(5, HIGH);
86     msdelay(50);
87     digitalWrite(5, LOW);
88     msdelay(50);
89     for (int i = 0; i < 2; i++) {
90         digitalWrite(3, HIGH);
91         digitalWrite(4, HIGH);
92         digitalWrite(5, HIGH);
93         msdelay(150);
94         digitalWrite(3, LOW);
95         digitalWrite(4, LOW);
96         digitalWrite(5, LOW);
97         msdelay(150);
98     }
```

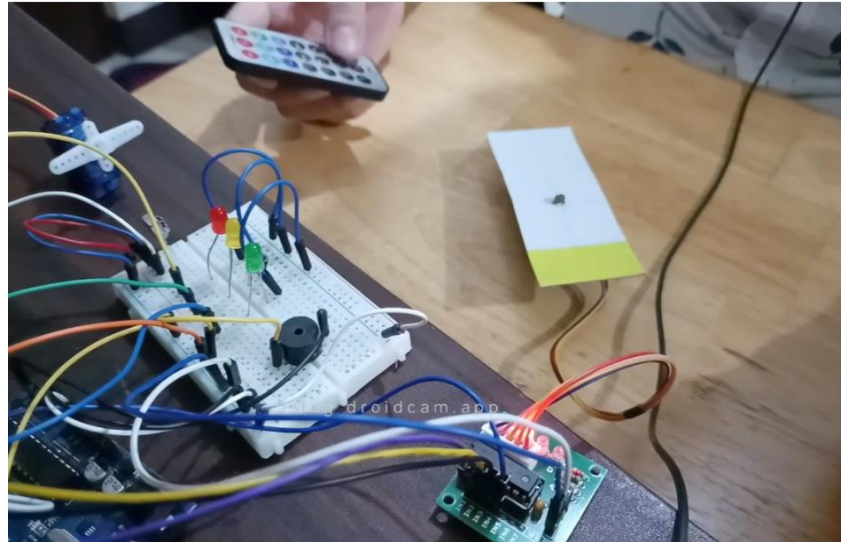
```
99     digitalWrite(5, HIGH);
100    msdelay(50);
101    digitalWrite(5, LOW);
102    msdelay(50);
103    digitalWrite(4, HIGH);
104    msdelay(50);
105    digitalWrite(4, LOW);
106    msdelay(50);
107    digitalWrite(3, HIGH);
108    msdelay(50);
109    digitalWrite(3, LOW);
110    msdelay(50);
111    for (int i = 0; i < 2; i++) {
112        digitalWrite(3, HIGH);
113        digitalWrite(4, HIGH);
114        digitalWrite(5, HIGH);
115        msdelay(150);
116        digitalWrite(3, LOW);
117        digitalWrite(4, LOW);
118        digitalWrite(5, LOW);
119        msdelay(150);
120    }
121 }
```

Figure 9. Codes for LED() function

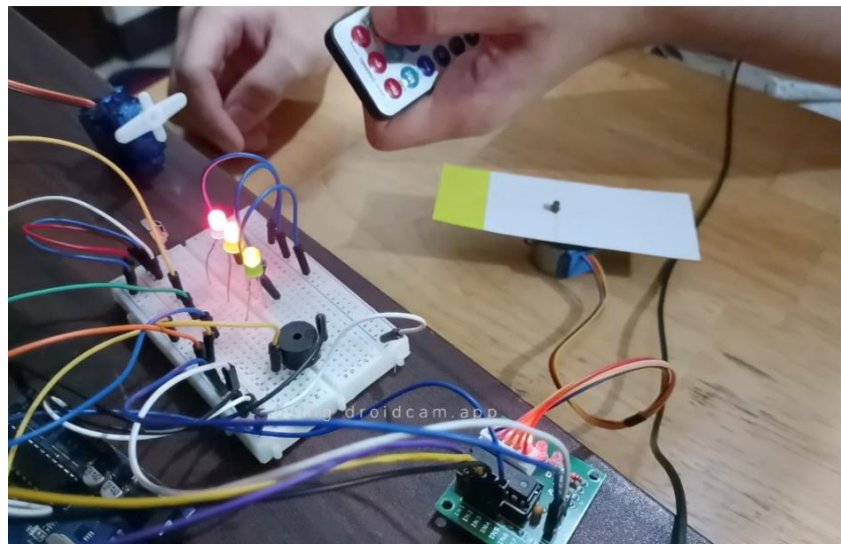
Lastly, to activate all of the components at once, the loop simply contains a repeat of all of the program and codes utilized for each of the components. Button 5 is assigned to the activation of all the components at once.

```
156     } else if (irval == 5) {  
157         while (irval != 10) {  
158             remote();  
159             if (irval == 10)  
160             {  
161                 break;  
162             }  
163             digitalWrite(3, HIGH);  
164             digitalWrite(4, HIGH);  
165             digitalWrite(5, HIGH);  
166             msdelay(50);  
167             if (irval == 10)  
168             {  
169                 break;  
170             }  
171             stepper.step(100);  
172             servo.write(180);  
173             playTone(1000, 500);  
174             remote();
```

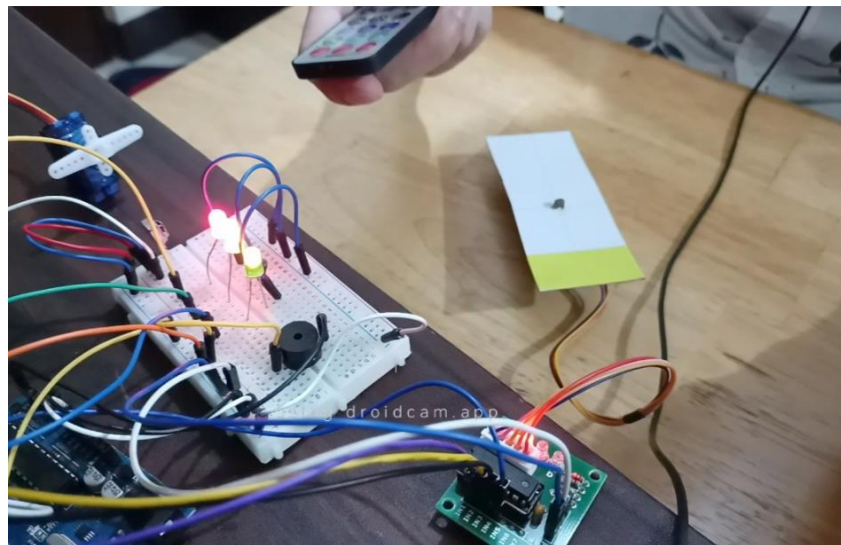
Figure 10. Activation of all Components



(a)



(b)



(c)

Figure 11. Sample Final Outputs of Experiment 4

CONCLUSION

To conclude, for the first objective, all components are successfully controlled through the use of buttons 1 - 4 for stepper motor, servo motor, buzzer, and LEDs, respectively, and each component can be stopped or reset using the pause/play button. Moreover, an additional function was included, wherein all of them will activate at once using button 5 as the trigger. The usage of microprocessors, logic circuits, and switching theory, as well as the lessons in CPE160P, was shown utilizing series connections, pin layouts, motors, buzzers, and output activation, as the second aim was met. Ultimately, to meet the third goal, the appliance controller was successfully built using the Arduino kit and

demonstrated in a basic handshaking by employing various variables, modules, and invoking such functions to develop an embedded system.